

Assignment No. 01

Generative AI

Fall 2026

Deadline: As Per GCR

*Sequence-to-Sequence Question Generation for Urdu
(built and trained from scratch)*

Submission Instructions:

- This is a group assignment.
- Each group should be exactly 2 members.
- **Plagiarism is strictly prohibited. Plagiarism among groups would be checked with tools.**
- **Please do not use any AI tool for direct copy / paste and do not copy from others.**
- **AI plagiarism using tools such as ChatGPT, Claude, Gemini, Copilot, Cursor, etc. will be marked straight zero.**
- **Submission carries no marks. Marks will be awarded based on viva.**
- Write a Medium Blog and a LinkedIn Post and submit their links.
- Develop the front end, and push all code to GitHub with commits from both team members. Submit Repo Link.
- Use Colab or Kaggle for GPU access.

Read the whole manual before you begin.

Contents

1	Overview	3
1.1	The problem	3
1.2	Constraints	3
1.3	Dataset: UQA	3
2	Instructions	4
2.1	Task 1 – Data preparation	4
2.2	Task 2 – Tokenizer	4
2.3	Task 3 – Model	4
2.4	Task 4 – Evaluation	4
2.5	Task 5 – Front end	5
2.6	Task 6 – Communication	5
3	Evaluation	5
3.1	Automatic metrics	5
3.2	Human evaluation	5
4	Results	5
4.1	Table 1 – Dataset statistics	6
4.2	Table 2 – Model configuration	6
4.3	Table 3 – Automatic metrics	6
4.4	Table 4 – Human evaluation (50 samples)	6
4.5	Figures	6
4.6	Qualitative samples	6
4.7	Discussion	6
	Appendix: Starter Code	8

1. Overview

1.1 The problem

Given an Urdu sentence with the *answer* marked inside it, generate the question that the answer responds to. This is what a teacher does when turning a textbook paragraph into practice questions; you will build a model that does it for them.

Input	An Urdu sentence with the answer span wrapped in <code><ans> ... </ans></code>
Output	An Urdu question whose answer is exactly the marked span

Example (shown in English): “*The Indus is about <ans>3,180 km</ans> long*” → “*How long is the Indus?*”

This is a **sequence-to-sequence** task: a variable-length input is mapped to a variable-length output with no fixed alignment, the same family as translation, summarisation and every modern generative language model.

1.2 Constraints

- **From scratch.** An RNN (LSTM/GRU) encoder–decoder with attention. No pretrained weights, no Transformers, no off-the-shelf seq2seq models.
- **Your own tokenizer.** Learn a subword vocabulary from the data with SentencePiece.
- **Shipped.** The model runs behind a small web front end.
- **Explained.** All marks come from the viva.

1.3 Dataset: UQA

UQA: Corpus for Urdu Question Answering (Arif, Farid, Athar and Raza, LREC-COLING 2024) translates SQuAD 2.0 into Urdu while preserving answer-span offsets. About 142k context–question–answer rows, including unanswerable ones you will drop.

Dataset	https://huggingface.co/datasets/uqa/UQA
Out-of-domain test	https://huggingface.co/datasets/uqa/Wiki-UQA
Code and paper	https://github.com/sameearif/UQA

Field	Type	Meaning
<code>context</code>	string	Urdu paragraph
<code>question</code>	string	Urdu question
<code>answers.text</code>	list[string]	answer span(s); empty if unanswerable
<code>answers.answer_start</code>	list[int]	character offset of the answer in <code>context</code>

Why sentence-level input?

A from-scratch model on ~90k examples cannot learn to read a 300-token paragraph. Using only the sentence that contains the answer is the setting of Du et al. (2017) and makes the task learnable.

2. Instructions

Complete the tasks in order. Every deliverable must exist in your GitHub repository. Starter code for Tasks 1–2 is in the Appendix.

2.1 Task 1 – Data preparation

- 1.1 Load `uqa/UQA`; keep only rows with a non-empty answer.
- 1.2 Find the sentence containing `answer_start` using character offsets. Sentence delimiters: `U+06D4` (Urdu full stop), `U+061F` (Urdu question mark), `!`.
- 1.3 Wrap the answer in `<ans> ... </ans>`; this sentence is the source, the `question` field is the target.
- 1.4 Drop pairs with source `> 60` or target `> 25` whitespace tokens.
- 1.5 Write `train.tsv` and `valid.tsv`; report counts and length histograms.

2.2 Task 2 – Tokenizer

- 2.1 Train `SentencePiece` (vocab 8k) on training sources and targets.
- 2.2 Register `<ans>`, `</ans>` as user-defined symbols; reserve `<pad>`, `<unk>`, `<s>`, `</s>`.
- 2.3 Show five tokenised examples and comment on how Urdu morphology is split.

2.3 Task 3 – Model

Implement **one** RNN encoder–decoder with attention from primitives (`nn.Embedding`, `nn.LSTM`/`nn.GRU`, `nn.Linear`, ...). No pretrained parts. Train it once; this is not a comparison study.

Encoder	2-layer bidirectional LSTM (or GRU)
Decoder	2-layer LSTM (or GRU) with Bahdanau or Luong attention
Sizes	embedding 256, hidden 512, dropout 0.3
Training	teacher forcing; cross-entropy that ignores padding; Adam
Decoding	greedy <i>and</i> beam search (beam 3–5)
Bookkeeping	save best checkpoint by validation loss; log loss per epoch; report parameter count

Compute

This trains in 1–2 hours on a free Colab or Kaggle GPU (batch 64, 10–15 epochs). Debug on a 10k-pair subset first. If loss does not fall after one epoch on the subset, the bug is in your code.

2.4 Task 4 – Evaluation

Evaluate the best checkpoint on UQA validation and on Wiki-UQA (same pipeline) following Section 3. Save `results/samples.tsv` (source, reference, greedy, beam) for at least 50 validation examples.

2.5 Task 5 – Front end

A minimal web UI (Streamlit, Gradio, Flask, ...) where a user pastes an Urdu sentence, marks the answer, and sees the greedy and beam outputs from *your* model. Screenshot in the README.

2.6 Task 6 – Communication

- **Medium blog** (800–1500 words): problem, design decisions, results, failure cases, one figure, three real outputs.
- **LinkedIn post** linking the blog and repository.
- **GitHub repository**: public, reproducible README, meaningful commits from *both* members.

Rules

No pretrained models, embeddings or checkpoints. AI assistants may explain concepts or debug errors; pasting code you cannot explain is plagiarism. Both members must be able to explain every part of the project.

3. Evaluation

3.1 Automatic metrics

Report on UQA validation and Wiki-UQA, for greedy and beam decoding:

BLEU-4	sacrebleu, corpus level, on detokenised output
ROUGE-L	F-measure, rouge-score package
Perplexity	$\exp(\text{mean validation cross-entropy})$ of the best checkpoint
<unk> rate	fraction of generated tokens that are <unk>

Expected range

Du et al. (2017) reached BLEU-4 ≈ 12 on English SQuAD. Expect roughly 6–13 on UQA validation and lower on Wiki-UQA. Near 0 means a bug; above 30 means train/validation leakage.

3.2 Human evaluation

Sample 50 validation outputs (fixed seed). Both members rate each independently, yes/no:

Fluency	grammatical, natural Urdu
Relevance	about the content of the source sentence
Answerability	the marked span answers the generated question

Report % yes per criterion and Cohen’s κ between members.

4. Results

Present the following in **results/** and in the blog, using these layouts.

4.1 Table 1 – Dataset statistics

	Train	Validation	Wiki-UQA
Rows in raw dataset			
Answerable rows			
Pairs after length filter			
Mean source / target length			

4.2 Table 2 – Model configuration

Encoder / decoder type (LSTM or GRU)
Layers / embedding / hidden size
Vocabulary size
Trainable parameters
Optimiser, learning rate, schedule
Batch size, epochs, wall-clock, GPU

4.3 Table 3 – Automatic metrics

Split	Decoding	BLEU-4	ROUGE-L	PPL	<unk> %
UQA valid	greedy				
UQA valid	beam ($k=...$)				
Wiki-UQA	greedy				
Wiki-UQA	beam ($k=...$)				

4.4 Table 4 – Human evaluation (50 samples)

	Fluency	Relevance	Answerability
Member 1 (% yes)			
Member 2 (% yes)			
Cohen's κ			

4.5 Figures

1. Training and validation loss per epoch.
2. Source/target length histograms.
3. One attention heat-map (source tokens $\times$ generated tokens).
4. Front-end screenshot.

4.6 Qualitative samples

Ten examples in the blog and `results/samples.tsv`: five good, five bad. For each bad one, name the failure type (wrong question word, hallucinated entity, <unk>, repetition, copied the sentence, ...).

4.7 Discussion

- Which question words does the model get right most often?

- Where does beam search help, and where does it hurt?
- Why does performance drop on Wiki-UQA, and what does that say about translated training data?

Appendix: Starter Code

Python 3.10 with `datasets` and `sentencepiece`. Urdu characters appear as Unicode escapes so the manual compiles anywhere; your code may use literal characters.

A.1 Environment

```
1 pip install datasets sentencepiece sacrebleu rouge-score torch
```

A.2 Load the dataset and inspect it

```
1 from datasets import load_dataset
2
3 ds = load_dataset("uqa/UQA")           # splits: train, validation
4 print(ds)
5
6 ex = ds["train"][0]
7 print(ex.keys())                       # id, title, context, question
8                                     , answers
9 print(ex["question"])
10 print(ex["answers"])                   # {'text': [...], '
11                                     answer_start': [...]}
12
13 n_total = len(ds["train"])
14 n_ans = sum(len(a["text"]) > 0 for a in ds["train"]["answers"])
15 print(f"train rows: {n_total}, answerable: {n_ans}")
```

A.3 Build sentence-level source/target pairs

```
1 import csv
2
3 ANS_OPEN, ANS_CLOSE = "<ans> ", "</ans> "
4 # Urdu full stop U+06D4, Urdu question mark U+061F, exclamation mark
5 SENT_DELIMS = "\u06D4\u061F!"
6
7 def split_sentences(text):
8     """Yield (start, end, sentence) with offsets into 'text'."""
9     start = 0
10    for i, ch in enumerate(text):
11        if ch in SENT_DELIMS:
12            yield start, i + 1, text[start:i + 1]
13            start = i + 1
14    if start < len(text):
15        yield start, len(text), text[start:]
16
17 def make_pair(example, max_src=60, max_tgt=25):
18     """Return (source, target) or None if the row is unusable."""
19     answers = example["answers"]
20     if len(answers["text"]) == 0:           # unanswerable -> skip
21         return None
22     a_start = answers["answer_start"][0]
23     a_text = answers["text"][0]
24     a_end = a_start + len(a_text)
25     context = example["context"]
26
27     for s, e, sent in split_sentences(context):
```

```

28     if s <= a_start < e:
29         rel = a_start - s                                # offset inside the
30         sentence
31         if sent[rel:rel + len(a_text)] != a_text:
32             return None                                    # offset mismatch ->
33             skip
34         src = (sent[:rel] + " " + ANS_OPEN + " " + a_text + " "
35               + ANS_CLOSE + " " + sent[rel + len(a_text):]).
36               strip()
37         src = " ".join(src.split())                       # normalise whitespace
38         tgt = " ".join(example["question"].split())
39         if len(src.split()) > max_src or len(tgt.split()) >
40             max_tgt:
41             return None
42         return src, tgt
43     return None
44
45 def build_split(split, out_path):
46     pairs = [p for p in map(make_pair, split) if p is not None]
47     with open(out_path, "w", encoding="utf-8", newline="") as f:
48         w = csv.writer(f, delimiter="\t", quoting=csv.QUOTE_NONE,
49                       escapechar="\\"")
50         w.writerows(pairs)
51         print(f"{out_path}: {len(pairs)} pairs")
52     return pairs
53
54 train_pairs = build_split(ds["train"], "train.tsv")
55 valid_pairs = build_split(ds["validation"], "valid.tsv")

```

A.4 Train the SentencePiece tokenizer

```

1  import sentencepiece as spm
2
3  # One line per sentence: sources and targets from the TRAIN split
4  # only
5  with open("sp_corpus.txt", "w", encoding="utf-8") as f:
6      for src, tgt in train_pairs:
7          f.write(src + "\n" + tgt + "\n")
8
9  spm.SentencePieceTrainer.train(
10     input="sp_corpus.txt",
11     model_prefix="ur_sp",
12     vocab_size=8000,
13     model_type="unigram",
14     character_coverage=1.0,           # keep every Urdu character
15     user_defined_symbols=[ANS_OPEN, ANS_CLOSE],
16     pad_id=0, unk_id=1, bos_id=2, eos_id=3,
17 )
18
19 sp = spm.SentencePieceProcessor(model_file="ur_sp.model")
20 PAD, UNK, BOS, EOS = 0, 1, 2, 3
21
22 src, tgt = train_pairs[0]
23 print(sp.encode(src, out_type=str))    # pieces
24 print(sp.encode(tgt))                 # ids
25 print(sp.decode(sp.encode(tgt)) == tgt) # round-trip check

```

A.5 Scoring

```
1 import sacrebleu
2 from rouge_score import rouge_scorer
3
4 def score(hyps, refs):
5     bleu = sacrebleu.corpus_bleu(hyps, [refs]).score
6     scorer = rouge_scorer.RougeScorer(["rougeL"], use_stemmer=False)
7     rl = sum(scorer.score(r, h)["rougeL"].fmeasure
8             for h, r in zip(hyps, refs)) / len(refs)
9     unk_rate = sum(h.count("\u2047") for h in hyps) / max(
10         1, sum(len(h.split()) for h in hyps)) # sp decodes <unk> as
11         U+2047
12     return {"BLEU-4": bleu, "ROUGE-L": rl, "unk_rate": unk_rate}
```

Good luck. Build something a teacher would actually use.